

快手前端AI面试

一面（技术基础）

1. 简单介绍一下事件循环机制，特别是微任务和宏任务的执行顺序。

难度：★

考点：JavaScript 异步执行机制

答案要点：

事件循环是 JavaScript 处理异步操作的核心机制，通过任务队列的循环调度确保单线程下的非阻塞执行。

- 执行栈清空后，优先检查并清空微任务队列（Microtask Queue）
- 微任务清空后，取出一个宏任务（Macrotask）执行
- 常见的微任务包括 Promise.then、MutationObserver、process.nextTick
- 常见的宏任务包括 setTimeout、setInterval、I/O、UI 渲染

常见坑：

- 误认为 Promise 构造函数内部是异步的（其实是同步执行）
- 忽略了 await 后面的代码会被包装成微任务

追问：

- 如果在微任务里又创建了微任务，会阻塞后续的宏任务吗？

2. 在 AI 应用中经常用到流式输出，请说明 SSE（Server-Sent Events）和 WebSocket 的区别。

难度：★★

考点：网络协议、AI 流式传输

答案要点：

SSE 是一种基于 HTTP 的单向推送技术，非常适合 LLM 这种服务端持续输出、客户端只读的场景。

- 通信方向：SSE 是单向的（服务端到客户端），WebSocket 是全双工双向通信
- 协议基础：SSE 基于标准 HTTP 协议，WebSocket 是独立的协议（需握手升级）
- 断线重连：SSE 内置了自动重连机制，WebSocket 需要手动实现
- 数据格式：SSE 默认传输文本流，WebSocket 支持二进制和文本

常见坑：

- HTTP/1.1 下 SSE 有 6 个并发连接限制，需配合 HTTP/2 使用

- 忘记在服务端设置 Cache-Control: no-cache

追问：

- 为什么目前大多数大模型对话接口首选 SSE 而不是 WebSocket?

3. 如何实现一个带有并发控制的 Promise 控制器？

难度：★★

考点：异步编程、手写代码能力

答案要点：

并发控制的核心是维护一个执行中的任务池，当任务完成后自动触发下一个待执行任务。

- 维护一个待执行队列和当前正在运行的任务计数器
- 每次调用 add 时，判断计数器是否达到上限，未达到则直接执行
- 执行逻辑中使用 finally 确保无论成功失败都能触发下一个任务
- 返回一个 Promise，以便外部感知当前任务的完成状态

常见坑：

- 任务执行失败后没有释放计数器导致后续任务永久挂起
- 没有正确处理返回值的传递

代码示例：

代码块

```
1  class PromisePool {
2    constructor(max) {
3      this.max = max;
4      this.count = 0;
5      this.queue = [];
6    }
7    add(fn) {
8      return new Promise((resolve, reject) => {
9        this.queue.push({ fn, resolve, reject });
10       this.run();
11     });
12   }
13   run() {
14     while (this.count < this.max && this.queue.length) {
15       this.count++;
16       const { fn, resolve, reject } = this.queue.shift();
17       fn().then(resolve).catch(reject).finally(() => {
18         this.count--;
19         this.run();
20       });
21     }
22   }
23 }
```

4. 谈谈 TypeScript 中的泛型，以及 extends 关键字在泛型中的作用。

难度：★

考点：TypeScript 基础

答案要点：

泛型是指在定义函数或类时不预先指定具体类型，而在使用时再指定类型的一种特性。

- 提高代码复用性：一套逻辑可以适配多种类型
- 类型约束：使用 extends 限制泛型必须符合某种结构
- 条件类型：结合 extends 实现类型分发 ($T \text{ extends } U ? X : Y$)
- 自动推断：编译器能根据输入参数自动推断泛型类型

常见坑：

- 泛型过度使用导致代码可读性下降
- 忽略了泛型默认值的设置

追问：

- 如何用泛型实现一个 Pick 工具类型？

5. CSS 如何实现一个类似 ChatGPT 对话框的自动滚动到底部效果？

难度：★

考点：CSS 布局、交互体验

答案要点：

实现对话框自动滚动通常结合 CSS 的布局特性和 JavaScript 的 DOM 操作。

- 容器设置 overflow-y: auto 开启滚动
- 使用 flex-direction: column-reverse 可以让内容从底部开始排列
- 现代浏览器可以使用 scroll-behavior: smooth 实现平滑滚动
- JavaScript 侧通过设置 scrollTop = scrollHeight 来强制触底

常见坑：

- 忽略了用户手动向上滚动时的干扰，不应强制打断用户阅读
- 图片或 Markdown 渲染延迟导致高度计算不准

追问：

- 如果内容非常多，如何优化长列表的渲染性能？

6. 算法题：给定一个字符串，判断其中的括号（包括大中小括号）是否合法有效。

难度：★

考点：栈结构、字符串处理

答案要点：

利用栈（Stack）后进先出的特性，遍历字符串进行匹配。

- 遇到左括号入栈
- 遇到右括号，检查栈顶是否为对应的左括号
- 匹配则出栈，不匹配或栈为空则返回 false
- 遍历结束后，若栈为空则说明全部匹配成功

常见坑：

- 字符串为空或只有一个字符的边界处理
- 括号类型不匹配的判断逻辑

代码示例：

代码块

```
1  function isValid(s) {
2    const stack = [];
3    const map = { '(': ')', '[': ']', '{': '}' };
4    for (let char of s) {
5      if (map[char]) {
6        if (stack.pop() !== map[char]) return false;
7      } else {
8        stack.push(char);
9      }
10   }
11   return stack.length === 0;
12 }
```

二面（深入原理与项目）

1. React 的 Fiber 架构解决了什么问题？在 AI 聊天这种高频更新场景下有什么优势？

难度：★★★

考点：React 核心原理、性能优化

答案要点：

Fiber 是 React 16 引入的重写架构，旨在通过时间分片将同步不可中断的更新变为异步可中断。

- 解决痛点：避免长耗时的 JavaScript 计算阻塞浏览器主线程造成卡顿
- 核心机制：将 Reconciliation 过程拆分为多个小单元（Fiber）

- 优先级调度：根据任务紧急程度（如用户输入高于数据流更新）分配执行权
- AI 场景优势：在流式输出大量文字时，保证输入框和 UI 交互依然流畅

常见坑：

- 误以为 Fiber 提高了计算速度（实际上增加了计算量，只是提高了响应性）
- 混淆了 Render 阶段（可中断）和 Commit 阶段（不可中断）

追问：

- useTransition 在 AI 应用中可以怎么使用？

2. 在处理 AI 流式数据（Streaming）时，前端如何优雅地处理 Markdown 渲染和代码高亮？

难度：★★

考点：流式渲染、第三方库集成

答案要点：

流式渲染要求前端在数据不完整的情况下也能正确解析并展示已到达的内容。

- 增量解析：使用支持流式的 Markdown 解析器（如 markdown-it 或 micromark）
- 状态维护：前端缓存已收到的 buffer，每次追加后重新渲染或局部更新
- 代码高亮：使用 Prism.js 或 lowlight，注意在流式过程中处理未闭合的代码块
- 性能优化：避免全量 DOM 替换，使用 Virtual DOM 或增量更新策略

常见坑：

- 标签未闭合导致的渲染错乱（如加粗、代码块标签）
- 频繁渲染导致的 CPU 占用过高

追问：

- 如果流式输出中包含 Latex 公式，你会如何处理渲染时机？

3. 谈谈你对 RAG（检索增强生成）流程的理解，前端在其中可以做哪些工作？

难度：★★

考点：AI 业务流程、前端边界扩展

答案要点：

RAG 通过在生成前检索外部知识库来缓解大模型的幻觉问题，前端主要负责交互和链路展示。

- 知识库管理：前端实现文档上传、切片预览、向量化进度展示
- 引用溯源：在 AI 回复中标注引用出处，点击可跳转至原文档对应位置
- 检索反馈：允许用户对检索到的片段进行“相关性”评价，辅助后端优化
- 调试工具：展示检索到的原始 Context，帮助开发者排查 Prompt 问题

常见坑：

- 引用链接失效或文档权限控制不当

- 检索片段过长导致前端 UI 撑破

追问：

- 如何设计一个展示知识库引用来源的 UI 组件？

4. 如何优化 Prompt 的构建过程？前端如何封装 Prompt 模板？

难度：★★

考点：Prompt Engineering、工程化思维

答案要点：

Prompt 工程化是将业务逻辑与模型指令分离的过程，前端通常负责动态参数的注入。

- 模板化：使用类似 Mustache 的语法定义 Prompt 模板，分离变量与指令
- 角色管理：封装 System Prompt、User Prompt 的构造函数
- 长度控制：在前端预估 Token 长度，必要时进行截断或压缩
- 版本控制：将 Prompt 视为代码资产，支持多版本灰度测试

常见坑：

- 直接在业务代码里硬编码长字符串 Prompt
- 忽略了用户输入中的特殊字符可能导致的 Prompt 注入攻击

追问：

- 前端如何计算 Token 数量？（提到 tiktoken 或类似库）

5. 你们项目里是如何做前端性能优化的？请结合 AI 应用的特点聊聊。

难度：★★

考点：性能优化实践

答案要点：

AI 应用的性能优化除了常规的加载优化，更侧重于感知性能和长列表性能。

- 感知性能：利用流式输出（TTFT 优化）、骨架屏、打字机效果缓解焦虑
- 长列表：AI 对话往往很长，必须使用虚拟列表（Virtual List）处理上千条消息
- 资源加载：大模型相关的 Web Worker 或 WASM 模块（如分词器）采用异步加载
- 缓存策略：对历史对话记录进行 IndexedDB 本地持久化

常见坑：

- 虚拟列表在高度动态（Markdown 渲染后高度变化）时的抖动问题
- 内存泄漏：长对话场景下未及时销毁的监听器或大对象

追问：

- 如何监控 AI 响应的“首字延迟”（Time to First Token）？

6. 介绍一下 Web Worker 的应用场景，在 AI 前端应用中它能解决什么问题？

难度：★★

考点：多线程编程、计算密集型任务

答案要点：

Web Worker 为 JavaScript 提供了多线程能力，可以将耗时计算移出主线程。

- 文本处理：在前端进行大规模文本清洗、分词或 Embedding 计算
- 语法高亮：对超长 Markdown 代码块进行高亮解析，避免 UI 卡死
- 数据转换：处理向量数据库返回的大量 JSON 数据
- 离线模型：运行轻量级的 Transformers.js 等前端推理模型

常见坑：

- Worker 内部无法直接操作 DOM
- 频繁的数据通信（PostMessage）带来的序列化开销

追问：

- 什么是 Transferable Objects？它在 Worker 通信中有什么优势？

7. 如果 AI 返回的内容包含恶意脚本，前端如何防范 XSS 攻击？

难度：★★

考点：安全、内容安全策略

答案要点：

AI 生成的内容具有不可预测性，必须经过严格的清洗和过滤。

- 使用成熟的 Sanitizer 库（如 DOMPurify）对渲染前的 HTML 进行清洗
- 在 Markdown 解析阶段禁用危险标签（如 script, iframe）
- 配置严格的 CSP（Content Security Policy）策略
- 对代码块展示使用纯文本渲染，不执行其中的任何逻辑

常见坑：

- 误以为 Markdown 默认就是安全的
- 在“复制到剪贴板”功能中忽略了隐藏字符攻击

追问：

- 如果要支持 AI 生成并运行 React 组件（类似 Artifacts），该如何保证安全？

8. 手写题：实现一个简单的 EventBus（发布订阅模式）。

难度：★★

考点：设计模式、基础功底

答案要点：

发布订阅模式是解耦组件间通信的常用手段，需支持监听、触发、只触发一次和取消监听。

- 使用 Map 存储事件名与回调函数的对应关系

- on 方法添加订阅，emit 方法触发执行
- off 方法移除特定回调，once 方法执行后自动移除
- 注意处理回调函数不存在或重复订阅的情况

代码示例：

代码块

```

1  class EventBus {
2    constructor() {
3      this.events = {};
4    }
5    on(name, fn) {
6      if (!this.events[name]) this.events[name] = [];
7      this.events[name].push(fn);
8    }
9    emit(name, ...args) {
10     if (this.events[name]) {
11       this.events[name].forEach(fn => fn(...args));
12     }
13   }
14   off(name, fn) {
15     if (this.events[name]) {
16       this.events[name] = this.events[name].filter(f => f !== fn);
17     }
18   }
19 }

```

三面（架构与综合能力）

1. 如果让你从零设计一个 AI Agent 的前端架构，你会考虑哪些核心模块？

难度：★★★

考点：系统设计、架构思维

答案要点：

AI Agent 前端不仅是聊天框，更是一个复杂的交互系统，需要处理状态、工具和反馈。

- 状态管理中心：管理对话上下文、Token 消耗、Agent 运行状态（思考中/执行中）
- 协议适配层：统一封装 SSE、WebSocket 或长轮询，支持模型切换
- 交互组件库：包含 Markdown 渲染、图表展示、文件预览及 Tool Call 交互组件
- 插件/工具系统：定义 Agent 调用前端能力的规范（如修改页面样式、触发搜索）
- 监控与反馈：全链路 Trace ID 追踪、用户点赞/点踩、幻觉纠错上报

常见坑：

- 架构过于耦合，导致更换模型或增加工具时需要大规模重构
- 忽略了多轮对话中 Context 窗口的管理

追问：

- 如何处理 Agent 执行长任务时的超时和中断恢复？

2. 在 AI 应用中，前端如何配合后端实现高效的“引用溯源”功能？

难度：★★★

考点：RAG 落地、前后端协作

答案要点：

引用溯源需要打通从原始文档到向量片段再到 AI 生成内容的完整链路。

- 数据结构设计：后端在返回流式文本时，通过特殊占位符（如 `[^1]`）或 Metadata 携带来源 ID
- 预取机制：前端解析到来源 ID 后，异步请求该片段的上下文或原文位置
- 交互设计：点击引用标签弹出 Popover 显示原文，并高亮对应文本
- 深度链接：支持直接定位到 PDF 的页码或长文档的特定偏移量

常见坑：

- 文本切片导致语义断裂，用户看到的引用片段无法理解
- 多个引用指向同一文档时的 UI 堆叠问题

追问：

- 如果文档是图片或视频，溯源功能该如何演进？

3. 谈谈你在项目中遇到的最难的技術挑战是什么？你是如何解决的？

难度：★★★

考点：问题解决能力、项目深度

答案要点：

（此题为开放性回答，建议按 STAR 原则组织）

- 背景：描述一个具体的 AI 场景（如：流式渲染导致的页面极度卡顿）
- 任务：明确目标（如：在不影响打字机效果的前提下降低 CPU 占用）
- 行动：采取的措施（如：引入 `RequestAnimationFrame` 节流渲染、使用 Web Worker 解析 Markdown、优化 DOM 树层级）
- 结果：量化指标（如：FPS 从 20 提升到 60，首屏加载时间降低 30%）

常见坑：

- 描述过于平淡，体现不出“难”在哪里
- 解决方案过于常规，没有体现出深度思考

4. 如何评估一个 AI 前端应用的用户体验？有哪些关键指标？

难度：★★

考点：产品思维、质量保障

答案要点：

AI 应用的体验评估不仅看性能，还要看交互的确定性和智能感。

- 响应性指标：TTFT（首字延迟）、响应成功率、流式输出的平滑度
- 交互质量：Prompt 提示词的准确性、Tool Call 的成功率、反馈机制的覆盖率
- 业务指标：对话完成率、用户留存、点赞/点踩比（满意度）
- 异常处理：模型幻觉时的提示、网络中断后的断点续传能力

常见坑：

- 只关注技术指标，忽略了 AI 回复质量对体验的决定性影响
- 缺乏对长尾 Case 的监控

5. 向量数据库（Vector DB）在前端有应用场景吗？

难度：★★★

考点：技术视野、前沿探索

答案要点：

虽然向量数据库多在后端，但前端本地向量化和检索正在成为可能。

- 本地搜索：利用 Wasm 版的向量库（如 Voy）在浏览器端实现历史记录语义搜索
- 隐私保护：敏感数据在本地向量化并检索，无需上传服务端
- 性能加速：对高频问题在前端进行语义匹配，命中则直接返回本地缓存
- 离线 RAG：配合轻量级离线模型实现完全离线的 AI 助手

常见坑：

- 浏览器内存限制导致无法处理大规模向量
- 模型下载体积过大影响首屏加载

追问：

- 你了解哪些可以在浏览器运行的向量化模型？

6. 你对未来 1-2 年“AI + 前端”的发展趋势怎么看？

难度：★★

考点：行业洞察、职业规划

答案要点：

这是一个开放性话题，重在体现求职者对技术的敏感度和思考。

- 交互变革：从 GUI（图形界面）向 LUI（语言界面）和生成式 UI 转变
- 开发模式：AI 辅助编程（Copilot）向 AI 自动化重构、自动化测试演进
- 端侧智能：更多的小模型直接运行在浏览器端，降低延迟和成本

- 角色转变：前端工程师需要具备基本的模型调优和 Prompt 编排能力

常见坑：

- 观点过于悲观（AI 取代前端）或过于激进（完全抛弃传统开发）
- 缺乏具体的实例支撑

7. 在快手这样的大厂，如果让你负责一个 AI 创新项目，你会如何保证项目的快速迭代和稳定性？

难度：★★★

考点：工程化、项目管理

答案要点：

在大厂环境下，平衡创新速度与系统稳定性是核心挑战。

- 灰度发布：利用 A/B Testing 验证不同模型或 Prompt 的效果
- 监控告警：建立 AI 专项监控，包括 Token 成本、模型响应分布、异常捕获
- 模块化解耦：将 UI 层与 AI 逻辑层分离，确保模型升级不影响基础功能
- 自动化链路：构建 Prompt 自动化测试集，防止回归更新导致模型表现下降

常见坑：

- 忽视成本控制，导致项目因 Token 消耗过大被叫停
- 缺乏文档和规范，导致多人协作时 Prompt 逻辑混乱

8. 聊聊你对快手 AI 业务的理解，你觉得前端在短视频 AI 场景下有哪些发挥空间？

难度：★★

考点：业务匹配度、主动性

答案要点：

结合快手的业务场景（短视频、直播、电商）谈 AI 前端的落地。

- 内容创作：AI 辅助脚本生成、视频剪辑工具的前端实现
- 智能客服：电商场景下的 AI 导购、自动售后处理
- 交互增强：直播间的 AI 挂件、实时语音转文字交互
- 搜索优化：基于语义理解的视频搜索结果呈现

常见坑：

- 答非所问，完全不了解快手的业务背景
- 想法脱离实际，无法落地到前端技术栈